

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 8: HUSKYLENSAI VISION AND REAL-TIME COLOR DETECTION

SECTION 1

GROUP 9

INSTRUCTOR: DR WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV.

DR ZULKIFLI BIN ZAINAL ABIDIN

NO	NAME	MATRIC NO
1	NOR EFFENDI BIN ANUAR @ ANWAR	2310519
2	NUR AINA SAFFIYA BINTI MOHD TARMIZI	2411644
3	NUR ATIKAH BINTI KAMARUL BAHARIN	2319846

DATE OF SUBMISSION: 6 MAY 2026

TABLE OF CONTENTS

1.0 INTRODUCTION.....	3
2.0 MATERIALS AND EQUIPMENT.....	3
3.0 EXPERIMENTAL SETUP.....	4
3.1 Task 1.....	4
3.2 Task 2.....	5
4.0 METHODOLOGY.....	7
4.1 Task 1.....	7
4.1.1 Circuit Setup.....	7
4.1.2 Program Development and Uploading.....	7
4.1.3 Program Operation.....	8
4.1.4 Coding.....	8
4.2 Task 2.....	9
4.2.1 Circuit Setup.....	9
4.2.2 Program Development and Uploading.....	10
4.2.3 Program Operation.....	10
4.2.4 Coding.....	10
5.0 DATA COLLECTION.....	12
5.1 Task 1.....	12
5.2 Task 2.....	12
6.0 DATA ANALYSIS.....	13
6.1 Task 1.....	13
6.2 Task 2.....	13
7.0 RESULTS.....	14
7.1 Task 1.....	14
7.2 Task 2.....	15
8.0 DISCUSSION.....	16
8.1 Task 1.....	16
8.2 Task 2.....	16
9.0 CONCLUSION.....	16
10.0 RECOMMENDATIONS.....	17
10.1 Task 1.....	17
11.0 REFERENCES.....	18
ACKNOWLEDGEMENT.....	19
STUDENTS' DECLARATION.....	19

1.0 INTRODUCTION

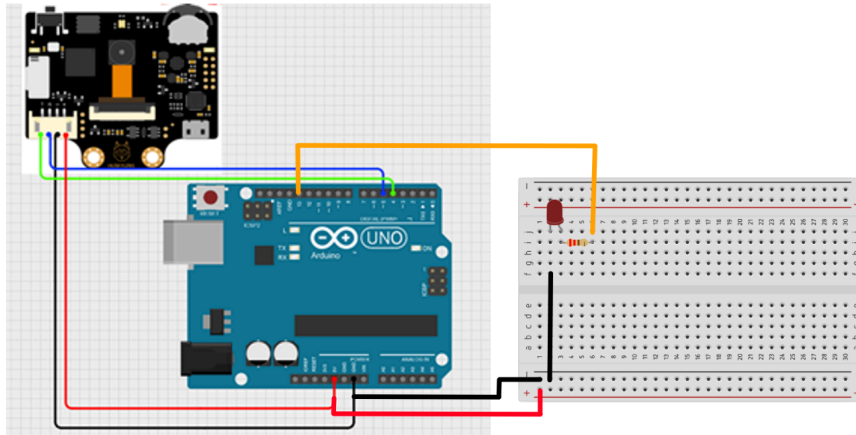
This experiment focuses on the integration of artificial intelligence with embedded systems using the Gravity HuskyLens AI Vision Camera and an Arduino Uno. The lab aims to bridge the gap between computer vision and physical actuation by teaching students how to interface hardware through UART serial communication and implement real-time object and color recognition. By utilizing machine learning algorithms built into the HuskyLens, the system can be trained to recognize specific visual patterns, which then trigger hardware responses such as standard LED switching and Pulse Width Modulation (PWM) control of an RGB LED module. This experiment demonstrates the fundamental principles of AI-driven mechatronics and sensor data parsing.

2.0 MATERIALS AND EQUIPMENT

- Arduino Uno + USB cable
- Gravity HuskyLens AI Vision Camera
- HW-479 RGB LED Module / 3x Leds + 3x 220 Ω resistors
- Jumper wires and breadboard

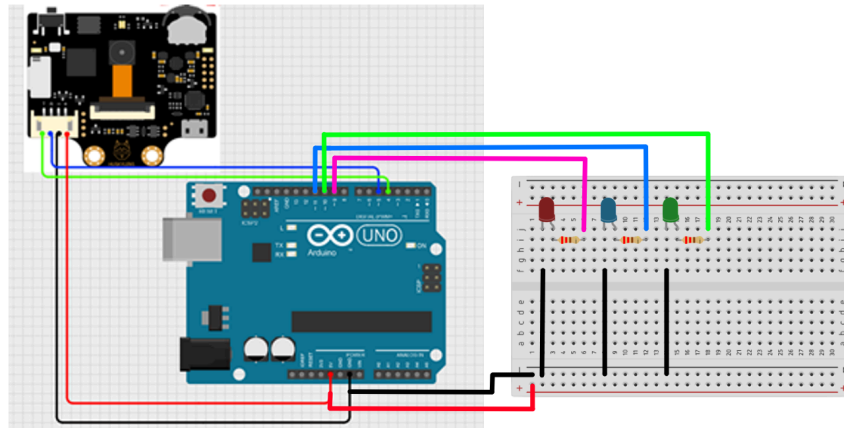
3.0 EXPERIMENTAL SETUP

3.1 Task 1



1. Connection between Arduino Uno and HuskeyLens AI Camera:
 - a. Connect the 5V pin on the Arduino Uno to the positive pin of the HuskyLens.
 - b. Connect the GND pin on the Arduino Uno to the negative pin of the HuckyLens.
 - c. Connect the D4 pin on the Arduino Uno to the pin most left side of the HuskyLens.
 - d. Connect the D5 pin on the Arduino Uno to the second pin from the left side of the HuskeyLens.
2. Connection between Arduino Uno and LED:
 - a. Connect the positive leg of the LED to the leg of 220Ω resistor.
 - b. Connect the other leg of the 220Ω resistor to the D8 pin on the Arduino.
 - c. Connect the negative leg of the LED to the GND pin on the Arduino.

3.2 Task 2



1. Connection between Arduino Uno and Breadboard:
 - a. Connect the 5V pin on the Arduino Uno to the positive terminal on the breadboard.
 - b. Connect the GND pin on the Arduino Uno to the negative terminal on the breadboard.
2. Connection between Arduino Uno and HuskeyLens AI Camera:
 - a. Connect the 5V pin on the Arduino Uno to the positive pin of the HuskyLens.
 - b. Connect the GND pin on the Arduino Uno to the negative pin of the HuskyLens.
 - c. Connect the D4 pin on the Arduino Uno to the pin most left side of the HuskyLens.
 - d. Connect the D5 pin on the Arduino Uno to the second pin from the left side of the HuskeyLens.

3. Connection between Arduino Uno and Red LED:

- a. Connect the positive leg of the red LED to the leg of 220Ω resistor.
- b. Connect the other leg of the 220Ω resistor to the D9 pin on the Arduino.
- c. Connect the negative leg of the red LED to the GND pin on the Arduino.

4. Connection between Arduino Uno and Green LED:

- a. Connect the positive leg of the green LED to the leg of 220Ω resistor.
- b. Connect the other leg of the 220Ω resistor to the D10 pin on the Arduino.
- c. Connect the negative leg of the green LED to the GND pin on the Arduino.

5. Connection between Arduino Uno and Blue LED:

- a. Connect the positive leg of the blue LED to the leg of 220Ω resistor.
- b. Connect the other leg of the 220Ω resistor to the D12 pin on the Arduino.
- c. Connect the negative leg of the blue LED to the GND pin on the Arduino.

4.0 METHODOLOGY

4.1 Task 1

4.1.1 Circuit Setup

- Connect the Gravity HuskyLens to the Arduino Uno. Map the VCC to 5V and GND to GND pins to provide stable power.
- Establish a UART serial connection by connecting the HuskyLens TX pin to Arduino Digital Pin 4 (RX) and the HuskyLens RX pin to Arduino Digital Pin 5 (TX).
- Install a standard LED on Digital Pin 13 of the Arduino to serve as the hardware response indicator.
- Access the HuskyLens onboard "General Settings" menu using the function dial. Manually select Serial 9600 as the protocol type to ensure compatibility with the SoftwareSerial baud rate.

4.1.2 Program Development and Uploading

- Import the HUSKYLENS.h and SoftwareSerial.h libraries into the Arduino IDE to handle data parsing and secondary serial communication.
- Develop a script using huskylens.request() to continuously poll the camera for detection data.
- Implement an if/else statement within the loop: if the camera detects ID 1, the system writes a HIGH signal to Pin 13; otherwise, it writes a LOW signal.
- Compile and flash the code to the Arduino Uno, ensuring the Serial Monitor is set to 115200 baud for debugging.

4.1.3 Program Operation

- Set the HuskyLens to "Object Recognition" mode. Point the camera at a target object and hold the "Learn" button to save it as ID 1.
- Verify that the physical LED on the breadboard toggles ON only when the trained object is within the camera's field of view.

4.1.4 Coding

```
#include "HUSKYLENS.h"

#include "SoftwareSerial.h"

HUSKYLENS huskylens;

SoftwareSerial mySerial(4, 5); // RX, TX

const int LED1 = 8;

void setup() {
    Serial.begin(115200);
    mySerial.begin(9600);

    pinMode(LED1, OUTPUT);

    // Wait for HuskyLens to connect
    while (!huskylens.begin(mySerial)) {
        Serial.println(F("Begin failed! Check wiring and Protocol
(Serial 9600)"));
        delay(500);
    }

    Serial.println("HuskyLens Ready!");
}
```



```

void loop() {

    if (!huskylens.request()) return; // Skip loop if no data

    // If the camera sees a trained object
    if (huskylens.available()) {

        HUSKYLENSResult result = huskylens.read();

        // Check if the result is a standard object block
        if (result.command == COMMAND_RETURN_BLOCK && result.ID ==
1) {

            Serial.print("Object ");

            Serial.print(result.ID);

            Serial.println(" detected!");

            digitalWrite(LED1,HIGH);

        }
        else {

            Serial.println("No Object Detected!");

            digitalWrite(LED1,LOW);

        }

    }

}

```

4.2 Task 2

4.2.1 Circuit Setup

- Connect the HW-479 RGB LED module to the Arduino. Connect the module's GND to the Arduino GND.
- Connect the Red (R), Green (G), and Blue (B) pins of the LED to PWM-enabled pins D9, D10, and D11 respectively.
- If using discrete LEDs, ensure each is equipped with a $220\ \Omega$ resistor on the

cathode side to prevent overcurrent.

4.2.2 Program Development and Uploading

- Modify the code to use a `switch(result.ID)` statement to handle multiple detection signals simultaneously.
- Define `analogWrite()` values (0–255) for each case. For ID 1 (Red), set D9 to 255; for ID 2 (Blue), set D11 to 255; and for ID 3 (Green), set D10 to 255.

4.2.3 Program Operation

- Switch the HuskyLens to "Color Recognition" mode. Train the sensor on Red, Blue, and Green samples in sequential order to assign them IDs 1, 2, and 3.
- Present the trained colors to the HuskyLens. Observe the RGB LED to ensure the physical light output color matches the "Actual Color" identified by the AI vision sensor.

4.2.4 Coding

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX

// DEFINE
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
  Serial.begin(115200);
  mySerial.begin(9600);

  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  // INITIAL LED OUTPUT
  setColor(0, 0, 0);
```

```

while (!huskylens.begin(mySerial)) {
    Serial.println(F("Begin failed!"));
    delay(500);
}
Serial.println("HuskyLens Ready!");
}

void loop() {
    if (!huskylens.request()) {
        setColor(0, 0, 0);
        return;
    }

    if (huskylens.available()) {
        HUSKYLENSResult result = huskylens.read();

        if (result.command == COMMAND_RETURN_BLOCK) {
            switch (result.ID) {
                case 1: // RED
                    Serial.println("Color RED is detected");
                    setColor(255, 0, 0);
                    break;
                case 2: // BLUE
                    Serial.println("Color BLUE is detected");
                    setColor(0, 255, 0);
                    break;
                case 3: // GREEN
                    Serial.println("Color GREEN is detected");
                    setColor(0, 0, 255);
                    break;
                default:
                    Serial.println("Unknown Color");
                    setColor(0, 0, 0);
                    break;
            }
        }
        else {
            // NO COLOUR DETECTED
            setColor(0, 0, 0);
        }
    }

    // FUNCTION TO SET RGB COLOUR
    void setColor(int redValue, int greenValue, int blueValue) {
        analogWrite(redPin, redValue);
        analogWrite(greenPin, greenValue);
        analogWrite(bluePin, blueValue);
    }
}

```

5.0 DATA COLLECTION

5.1 Task 1

Target Object	Husky Lens ID Detected	LED State (ON/OFF)
Trained Object	ID 1	ON
Untrained Object	None	OFF
No Object	None	OFF
Trained Object	ID 2	ON

5.2 Task 2

Actual Color Presented	Detected ID	LED Output Color
Red	ID 1	Red
Blue	ID 2	Blue
Green	ID 3	Green

6.0 DATA ANALYSIS

6.1 Task 1

The data for Task 1 demonstrates the HuskyLens's ability to successfully perform object recognition and trigger a hardware response via an Arduino Uno. When the "Trained Object" was presented to the camera, the system correctly identified it as ID 1 and ID 2, successfully triggering the LED to an ON state. Conversely, when the camera encountered an "Untrained Object" or "No Object," it returned a result of None, causing the LED to remain OFF. This confirms that the conditional logic (using if and else statements) effectively parsed the serial data from the camera to control the digital output. The consistency between the target object and the LED state proves that the HuskyLens was correctly configured to the Serial 9600 protocol and that the SoftwareSerial communication between the camera and the Arduino was stable.

6.2 Task 2

The analysis of Task 2 shows the successful integration of the HuskyLens in Color Recognition mode with a multi-output lighting system. The camera accurately mapped the "Actual Color Presented" to its predefined IDs: Red to ID 1, Blue to ID 2, and Green to ID 3. By using a switch statement to evaluate these IDs, the Arduino was able to drive the LED module to display the matching physical color. This indicates that the analogWrite (PWM) or digitalWrite logic correctly targeted pins to produce the intended visual feedback. While the trials for Red, Blue, and Green were 100% accurate, the reliability of these detections is sensitive to ambient lighting; any shifts in brightness or shadows could potentially cause the AI to misidentify these IDs, requiring re-calibration or training under consistent lighting conditions.

7.0 RESULTS

7.1 Task 1

The results for this experiment is a Husky lens successfully integrated with an arduino. When an object recognized by the husky lens is detected, then the system will trigger the COMMAND_RETURN_BLOCK. After that, the LED connected to pin 8 will turn on upon the detection of the recognized object. At the same time, the Serial monitor displayed “Object 1 detected!”. Besides that, when the object is removed from the view of the husky lens, then the LED will turn off.

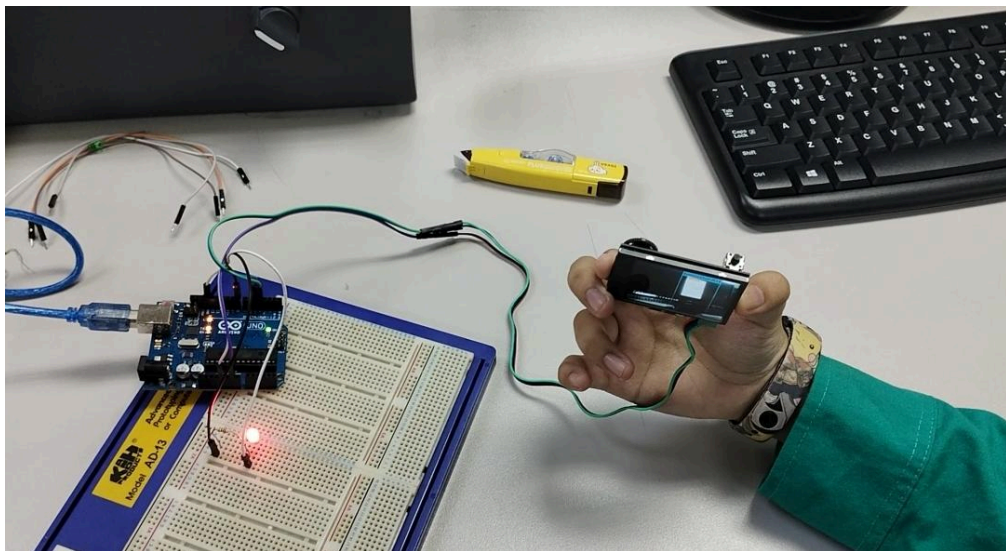


Diagram 7.1.1 show the HuskyLens detected an object and the LED turn on

7.2 Task 2

The Results for this experiment is that the system successfully learns multiple colours and can recognize it. First, the HuskyLens learns 3 colours : red as ID 1, green as ID2 and blue as ID 3. When it is done learning, if the HuskyLens detect the colour red, it will trigger the Red LED output, if the HuskyLens detect the colour blue, it will trigger the Blue LED output, and the same thing happens with the green colour. After that, if it detects colours other than red, blue and green, then the RGB LED light will turn off.



Diagram 7.2.1 show HuskyLens detect colour Red and RGB LED turn red

8.0 DISCUSSION

8.1 Task 1

The project demonstrates the ability of the husky lens to learn an object and can recognize it. The code utilizes the `huskylens.request()` function to poll data. The function of `“result.command == COMMAND_RETURN_BLOCK”` is to check the condition and it is critical to differentiate between data. After that, the HuskyLens must be manually set to serial 9600 in its onboard menu to match the SoftwareSerial in Arduino.

8.2 Task 2

In this Experiment, we use switch-case statements compared to if else statements in the previous experiment because the switch-case statements are much more efficient, readable and easy to debug. After that, during the experiment, we found out that when we show the colour green, the RGB LED activates the colour blue. While we show the colour blue to the HuskyLens, the RGB LED activates the colour green. In this case, we change the function of `“setColor()”` between green and blue. Thus, the problem is fixed and the system functions properly.

9.0 CONCLUSION

In conclusion, the experiment successfully demonstrated the integration of a Gravity HuskyLens AI Vision Camera with an Arduino Uno to create a responsive hardware system. Through the use of UART serial communication and the HUSKYLENS library, the system effectively parsed real-time visual data to trigger specific hardware outputs. For Task 1, the system achieved automated object recognition, successfully toggling a standard LED on

Digital Pin 8 whenever the trained "Object 1" was identified. In Task 2, the integration was expanded to multi-color recognition, where a switch statement was used to map detected color IDs (Red, Blue, and Green) to a multi-output LED system via PWM (analogWrite) and digitalWrite logic.

While the system proved highly reliable in identifying trained patterns, the accuracy was noted to be sensitive to ambient lighting conditions, confirming that while the objectives of AI-driven actuation were met, consistent lighting is a critical functional constraint for such mechatronic applications.

10.0 RECOMMENDATIONS

10.1 Task 1

For recommendation, we can use switch-case statements compared to if else statements to handle multiple IDs. For example, colour detection, we can make ID 1 as green, ID 2 as red and ID 3 as yellow colour. Other than that, we also can replace the simple if/else logic with a timer-based approach or a "timeout" function to prevent the LED from flickering if the camera momentarily loses sight of the object.

10.2 Task 2

For this experiment, we need to check the "setColor()" function and make sure the colour is correct. After that, adding a small delay in the "setColor()" function to prevent flickering when the HuskyLens transitions between the different colours.

Lastly, we can integrate servo motors along with the RGB LED to detect objects like paper and tins to make automatic recycle bins.

11.0 REFERENCES

HuskyLens. (n.d.). *GitHub - HuskyLens/HUSKYLENSArduino*. GitHub.

<https://github.com/HuskyLens/HUSKYLENSArduino>

Lee, J. (2025, September 8). *Getting Started with HuskeyLens & Arduino*. Cytron Technologies Malaysia.

<https://my.cytron.io/tutorial/getting-started-with-huskylens-and-arduino>

RootSaid - Robotics, Electronics & Technology. (2020, March 20). *HuskyLens Tutorial - Getting Started with Husky Lens - AI Vision Sensor* [Video]. YouTube.

<https://www.youtube.com/watch?v=HZxFj2u9k00>

ACKNOWLEDGEMENT

We would like to sincerely express our gratitudes to our instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal and the lab technician, for the help given during the task. The knowledge and suggestions are able to guide us throughout the progress of the task. We also would like to thank our classmates for their support and help which really give positive impacts to the progression of our task.

STUDENTS' DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed

unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: 

Read [/]

Name: NOR EFFENDI BIN ANUAR @ ANWAR

Understand [/]

Matric Number: 2310519

Agree [/]

Signature: 

Read [/]

Name: NUR AINA SAFFIYA BINTI MOHD TARMIZI

Understand [/]

Matric Number: 2411644

Agree [/]

Signature: 

Read [/]

Name: NUR ATIKAH BINTI KAMARUL BAHARIN

Understand [/]

Matric Number: 2319846

Agree [/]